Home
Library
Profile
Stories
Stats

Following
Dev Genius
The Useful Tech
Top Python Librari...
Generative AI
Realworld AI Use C...
Level Up Coding
Python in Plain En...
Deep concept
AI Mind
AI Quick Tips
More

How I Set Up Claude Code for Free (No Subscription, No Credit Card) — and What I Learned Along the Way

Prince Arora Follow 5 min read · Mar 16, 2026

57 10

A walkthrough of every decision, every mistake, and every concept that clicked.

. . .

I spent an afternoon trying to get Claude Code running without paying for an Anthropic subscription. What started as “this should take 20 minutes” turned into a deep dive through WSL internals, Python version conflicts, PowerShell profile quirks, and bash path escaping nightmares.

This is the article I wish existed before I started.

By the end, I had a fully working setup: Claude Code CLI running in VS Code, powered by Gemini 2.5 Flash as a free backend, with a clean bash terminal, a single `.env` file as the source of truth, and a setup ready to build agentic AI systems from scratch.

Here’s exactly how it works and why every decision was made.

. . .

First: What Even Is Claude Code?

Claude Code is Anthropic's AI coding assistant — a CLI tool (and VS Code extension) that lets you give natural language instructions and have an AI agent write, edit, and explain code in your project. Think of it as a pair programmer that lives in your terminal.

The catch: it's designed to work with an Anthropic subscription or API billing. Out of the box, it will ask you to log in and pay.

But here's the key insight that makes everything possible:

> **Claude Code is just a client that talks to the Anthropic API format. You can redirect it to any LLM that speaks that format.**

Set three environment variables — `ANTHROPIC_API_KEY`, `ANTHROPIC_BASE_URL`, and `ANTHROPIC_MODEL` — and Claude Code will happily talk to Gemini, Ollama, or any compatible backend instead.

. . .

The Free Options (and Which One Actually Works Without a Card)

Before settling on an approach, I mapped out every viable free option:

Ollama (local) — Completely free forever, runs models on your machine. The catch: you need serious hardware. 32GB+ RAM is recommended for models good enough to be genuinely useful for coding. With a 128MB GPU and no dedicated VRAM, this was off the table for me.

OpenRouter via claude-code-router — Access a rotating selection of free models (DeepSeek R1, Qwen, Llama) with no credit card required. Free accounts get 50 requests/day; add $10 in credits and it jumps to 1,000. Slightly more setup than Google AI Studio, but worth it if you want model variety. I chose Gemini purely for simplicity.

Ollama Cloud models — Ollama now supports `:cloud` variants that run on their servers rather than your machine. Still requires Ollama installed locally as the client/proxy.

Google AI Studio (Gemini) — Free tier, just a Google account. No credit card. No minimum top-up. Just sign in at aistudio.google.com, generate an API key, and you're done.

Winner: Google AI Studio. The Gemini 2.5 Flash

model is capable, the free tier is generous, and the barrier to entry is zero.

. . .

The Installation: A Mistake I Made

I installed the VS Code extension of Claude Code and assumed I was done. I wasn't.

The Claude Code VS Code extension and the CLI are not the same thing.

The VS Code extension is a UI panel — a sidebar, keyboard shortcuts, a nicer way to interact. It does nothing on its own. The actual engine is the CLI, installed via npm:

```
npm install -g @anthropic-ai/claude-code
```

The `-g` flag installs globally. Run it once, from anywhere, and `claude` becomes available in every terminal on your machine. You do not run this inside your project folder.

Verify it worked:

```
claude --version
```

> **Note:** Claude Code recently switched from npm to a native installer and will show a deprecation warning. As of this writing, there is a known bug where the native installer freezes in VS Code's integrated terminal on Windows. Stick with the npm install for now.

. . .

Pointing Claude Code at Gemini

With the CLI installed, the next step is redirecting it away from Anthropic's paid API. This is done entirely through environment variables — no code changes, no config files to hunt down.

The three variables you need:

```
ANTHROPIC_API_KEY=AIza-YOUR-GEMINI-KEY-HERE
ANTHROPIC_BASE_URL=https://generativelanguage.googleapis
ANTHROPIC_MODEL=gemini-2.5-flash
```

Get your free Gemini API key from aistudio.google.com — takes about 30 seconds.

The First-Run Login Screen Problem

Here's something the docs don't mention clearly: on first run, Claude Code shows a login screen even when these variables are set. It wants to authenticate with Anthropic.

The workaround: temporarily add a fourth variable:

```
ANTHROPIC_AUTH_TOKEN=AIza-YOUR-GEMINI-KEY-HERE
```

Same key, different variable name. This makes Claude Code detect it as a "custom API key" and ask if you want to use it — select Yes. Once your session is established, **remove** `ANTHROPIC_AUTH_TOKEN`. Keeping both variables causes a conflict warning on subsequent runs.

. . .

One .env File, Not Hardcoded Anywhere

Putting API keys directly in shell configs or VS Code settings is a bad habit. The right approach: a single `.env` file at your project root, loaded automatically.

```
ANTHROPIC_API_KEY=AIza-YOUR-GEMINI-KEY-HERE
```

```
ANTHROPIC_BASE_URL=https://generativelanguage.googleapis
ANTHROPIC_MODEL=gemini-2.5-flash
```

To auto-load it whenever a terminal opens, add a `Load-Env` function to your PowerShell profile. Open it with:

```
notepad $PROFILE
```

Then paste:

```
function Load-Env {
    $envFile = Join-Path (Get-Location) ".env"
    if (Test-Path $envFile) {
        Get-Content $envFile | ForEach-Object {
            if ($_ -match "^\s*([^#][^=]+)=(.+)$") {
                [System.Environment]::SetEnvironmentVar
            }
        }
        Write-Host ".env loaded" -ForegroundColor Green
    }
}
Load-Env
```

One gotcha: VS Code has its own PowerShell profile, separate from the regular PowerShell terminal. Always run `notepad $PROFILE` from inside VS Code's integrated terminal — not a standalone PowerShell window. They point to different files.

Also: always add `.env` to your `.gitignore`. Always.

```
echo '.env' >> .gitignore
```

. . .

Set up LiteLLM Proxy:

To connect claude code to gemini, we need a proxy layer in between. Refer the steps in link below 👇🏻

My Claude Code Gemini Setup Broke. Here's What Was Actually Happening.

My Claude Code Gemini Setup Broke. Here's What Was Actually Happening. A

prince-arora-aws.medium.com

. . .

That's It — You're Running

Once the `.env` is loaded, open your project folder in VS Code, activate your terminal, and type:

```
claude
```

You should see Claude Code launch with `gemini-2.5-flash` shown at the bottom of the panel. Run `/status` to confirm which model and endpoint it's connected to.

. . .

Key Takeaways

- Claude Code works completely free with Google AI Studio — just a Google account
- The VS Code extension and the CLI are separate things; install both
- Three environment variables are all it takes to redirect Claude Code to any backend
- `ANTHROPIC_AUTH_TOKEN` is a one-time trick to bypass the first-run login screen — remove it after
- A single `.env` file beats hardcoded credentials

everywhere

. . .

Next: building the first AI agent with this setup — a Hello Agent that makes its first API call.

. . .

Tags: AI, Claude, LLM, Python, Developer Tools, VS Code, Gemini, Free Tools, Productivity

Written by Prince Arora

122 followers · 10 following

IT professional | Technology Enthusiast | AWS | Machine Learning | Gen AI